Programação Orientada a Objetos

Arrays, Ponteiros e Funções em C++

Professor: Pedro Montoaneli
pedromontoaneli@teiacoltec.org

Sumário

- Arrays
  - Declaração
  - Arrays multidimensionais
  - Exemplos
- Ponteiros
  - Declaração e inicialização
  - Operadores
  - Arrays e ponteiros
  - Ponteiro para ponteiro
- Funções
  - Declaração
  - Tipos de passagem de parâmetros
  - Funções *inline*
- Exercícios

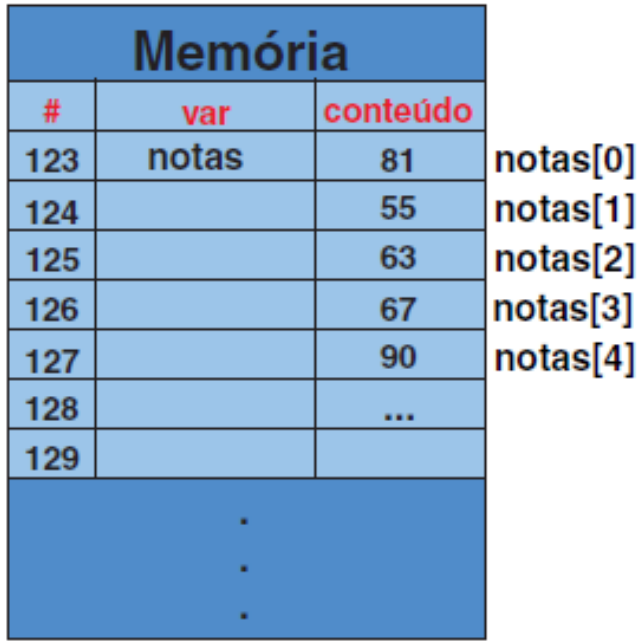
Declaração

- Um array é um grupo consecutivo de posições da memória em que todas são do mesmo tipo.
- A primeira posição do array é sempre o `0`, e não o `1`.

- Um array de n elementos tem seu primeiro elemento ocupando uma dada posição na memória, e os n-1 restantes ocupando as n-1 posições seguintes.

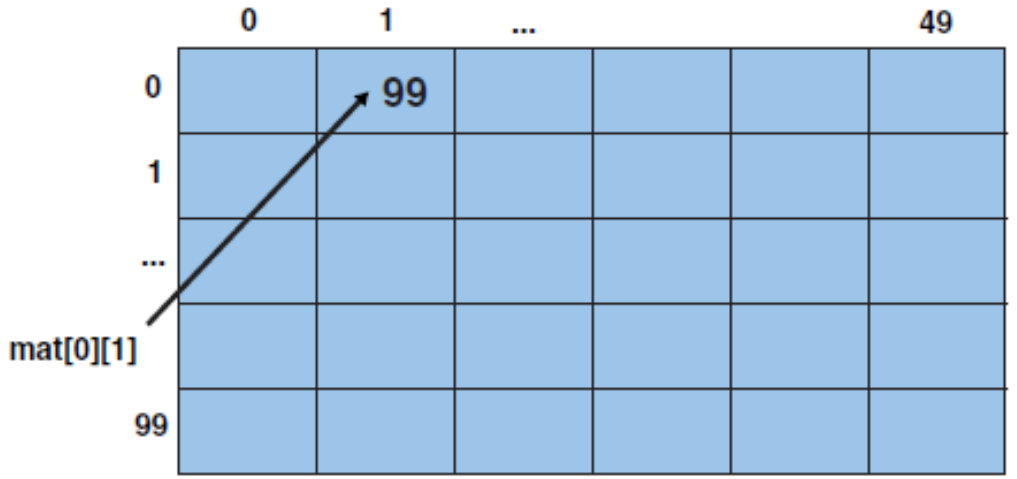
```
tipoDado nomeArray[tamanho]
```

```
float notas[100]
```



Arrays Multidimensionais

- Um array de uma dimensão pode ser tratado como uma lista de variáveis. Em duas dimensões, ele representa uma tabela de valores, que também é chamada de **matriz**.
- Matrizes se organizam em uma estrutura de linhas e colunas, e sua declaração é da forma:

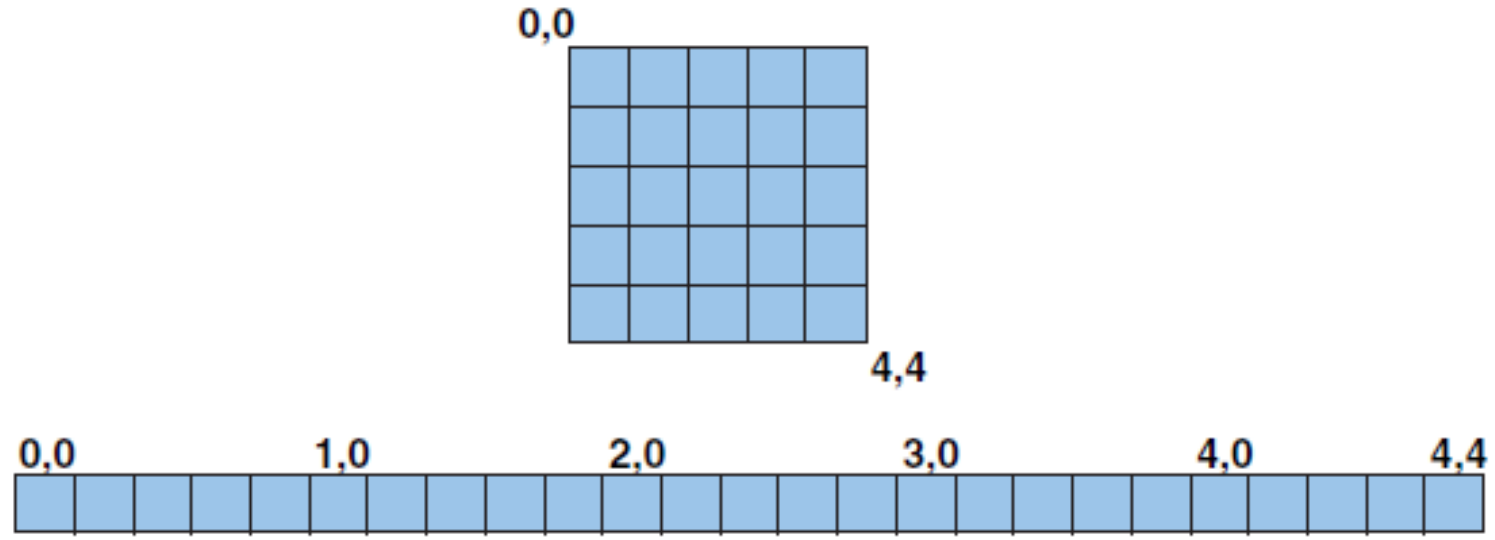



```
tipoDado nomeArray[numLinhas][numColunas]
```

```
int mat[100][50]
mat[0][1] = 99;
```

Arrays Multidimensionais

- Independentemente do número de dimensões que possuam, os dados dos arrays são sempre armazenados **linearmente** na memória.
- A dimensão do array que se “move” mais rapidamente é sempre a mais à **direita**.



Exemplos

1) Escreva um programa que leia do teclado seis valores inteiros e em seguida mostre na tela os valores lidos na ordem inversa.

2) Em uma turma com 3 alunos foram aplicadas duas provas. Escreva um programa que leia a nota de cada aluno em cada uma das provas. Em seguida, imprima na tela a média de cada um dos alunos.

Declaração e Inicialização

- Ponteiros são um tipo especial de variáveis que permitem armazenar endereços de memória em vez de dados numéricos ou caracteres.

```
tipoPonteiro *nomePonteiro
```

- Quando um ponteiro é declarado, ele não possui um endereço associado. Qualquer tentativa de uso desse ponteiro causa um comportamento indefinido no programa.

Memória		
#	var	conteúdo
119		
120	int *p	????
121		

Operadores

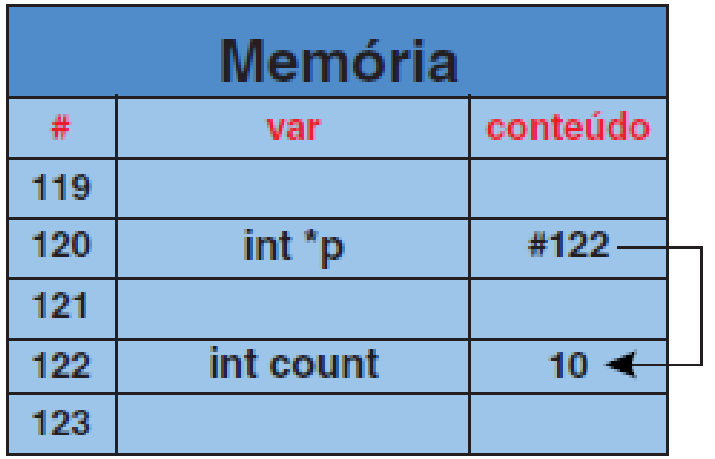
- Ao se trabalhar com ponteiros, duas tarefas básicas serão sempre executadas:
  - Acessar o endereço de memória de uma variável.
  - Acessar o conteúdo de um endereço de memória.

- Essas tarefas são realizadas pelos operadores `*` e `&`
  - `*p` == Conteúdo apontado por `p`.
  - `&x` == Endereço na memória da variável `x`.

```
int *p, x;
p = &x;
```

Operadores

```
1.#include <iostream>
2.using namespace std;
3.
4.int main(){
5.//Declara uma variável int contendo o valor 10
6.int count = 10;
7.//Declara um ponteiro para int
8.int *p;
9.//Atribui ao ponteiro o endereço da variável int
10.p = &count;
11.
12.cout << "Valor de p: " << p << endl;
13.cout << "Conteúdo apontado por p: " << *p << endl;
14.
15.return 0;
16.}
```



Ponteiros e Arrays

- O nome de um array é um ponteiro que aponta para a primeira posição do array.

```
1.#include <iostream>
2.using namespace std;
3.
4.int main(){
5.int vet[5]= {1,2,3,4,5};
6.int *p = vet;
7.int i;
8.for (i = 0;i < 5;i++)
9.cout << p[i] << endl;
10.return 0;
11.}
```

```
1.#include <iostream>
2.using namespace std;
3.
4.int main(){
5.int vet[5]= {1,2,3,4,5};
6.int *p = vet;
7.int i;
8.for (i = 0;i < 5;i++)
9.cout << *(p+i) << endl;
10.return 0;
11.}
```

- Em C/C++, a atribuição do endereço do array ao ponteiro pode ser feita de duas formas:

```
int *p = vet;
int *p = &vet[0];
```

Ponteiro para Ponteiro

- O C/C++ permitem criar ponteiros com diferentes níveis de apontamento, isto é, ponteiros que apontam para outros ponteiros.

```
tipoPonteiro **nomePonteiro
```

```
1.#include <iostream>
2.using namespace std;
3.
4.int main(){
5.int x = 10;
6.int *p = &x;
7.int **p2 = &p;
8.cout << “Endereco em p2: " << p << endl;
9.cout << “Conteudo em *p2: " << *p2 << endl;
10.cout << “Conteudo em **p2: " << **p2 << endl;
11.
12.return 0;
13.}
```



Declaração

- Programas grandes e complexos são construídos bloco a bloco com a ajuda de funções.
- O uso de funções evita a cópia desnecessária de trechos de código que realizam a mesma tarefa, diminuindo assim o tamanho do programa e a ocorrência de erros.

```
tipoRetornado nomeFuncao(listaParametros){
        declarações;
        return parametroRetornado;
}
```

Declaração

- Em geral, as funções são declaradas antes da `main()`.
- Pode-se também declarar uma função depois da `main()`. Nesse caso, épreciso declarar antes o protótipo da função.
- O protótipo de uma função é uma declaração de função que omite o corpo mas especifica o seu nome, tipo de retorno e lista de parâmetros.

```
1.#include <iostream>
2.using namespace std;
3.
4.int Square (int a);
5.
6.int main(){
7.//Código da main
8.return 0;
9.}
10.int Square (int a){
11.return (a*a);
12.}
```

Funções *inline*

- O qualificador `inline` diz ao compilador para criar uma cópia do código da função no seu lugar (quando apropriado) para evitar uma chamada de função.

- Utilizar funções *inline* pode reduzir o tempo de execução, mas pode aumentar o tamanho do programa.

```
1.#include <iostream>
2.using namespace std;
3.
4.inline float cube(float side )
5.{
6.    return side * side * side;
7.}
8.
9.int main(){
10.    double sideValue;
11.    cout << “Enter the side length of your cube: “;
12.    cin >> sideValue;
13.    cout << “Volume of cube with side “<< sideValue << “
is “ << cube( sideValue ) << endl;
14.    return 0;
15.}
```

Tipos de Passagem por Parâmetro

```
1.#include <iostream>
2.using namespace std;
3.
4.void Troca(int a,int b){
5.int temp;
6.temp = a;
7.a = b;
8.b = temp;
9.cout << “Dentro: " << a << "e " << b << endl;
10.}
11.int main(){
12.int x = 2;
13.int y = 3;
14.cout << “Antes: " << x << "e " << y << endl;
15.Troca(x,y);
16.cout << “Depois: " << x << "e " << y << endl;
17.
18.return 0;
19.}
```

```
1.#include <iostream>
2.using namespace std;
3.
4.void Troca(int& a,int& b){
5.int temp;
6.temp = a;
7.a = b;
8.b = temp;
9.cout << “Dentro: " << a << "e " << b << endl;
10.}
11.int main(){
12.int x = 2;
13.int y = 3;
14.cout << “Antes: " << x << "e " << y << endl;
15.Troca(x,y);
16.cout << “Depois: " << x << "e " << y << endl;
17.
18.return 0;
19.}
```

Exercícios

Exercício 1:

Escreva uma função `dobrar(int& n)` que dobra o valor recebido, alterando o original, e uma função `dobrarArray` que recebe um array de inteiros e seu tamanho e dobra todos os elementos. No `main`, crie um array, imprima antes, chame `dobrarArray`, e imprima depois para mostrar que o array original mudou.

Exercício 2:

Escreva uma função `encontrarMaior` que recebe um array de inteiros e o tamanho, e devolve o endereço do maior elemento, ou seja, retorna um ponteiro. No `main`, chame a função e use o ponteiro recebido para imprimir o maior valor.